



```
render() {  
  return (  
    <React.Fragment>  
      <div className="py-5">  
        <div className="container">  
          <Title name="our" title="product">  
            <div className="row">  
              <ProductConsumer>  
                {(value) => {  
                  console.log(value)  
                }}  
              </ProductConsumer>  
            </div>  
          </div>  
        </div>  
      </React.Fragment>  
    )  
  )  
}
```

# Algoritmo para calcular cidades mais distantes

Apresentado por  
**Antonio A. do Nascimento**

# Problema

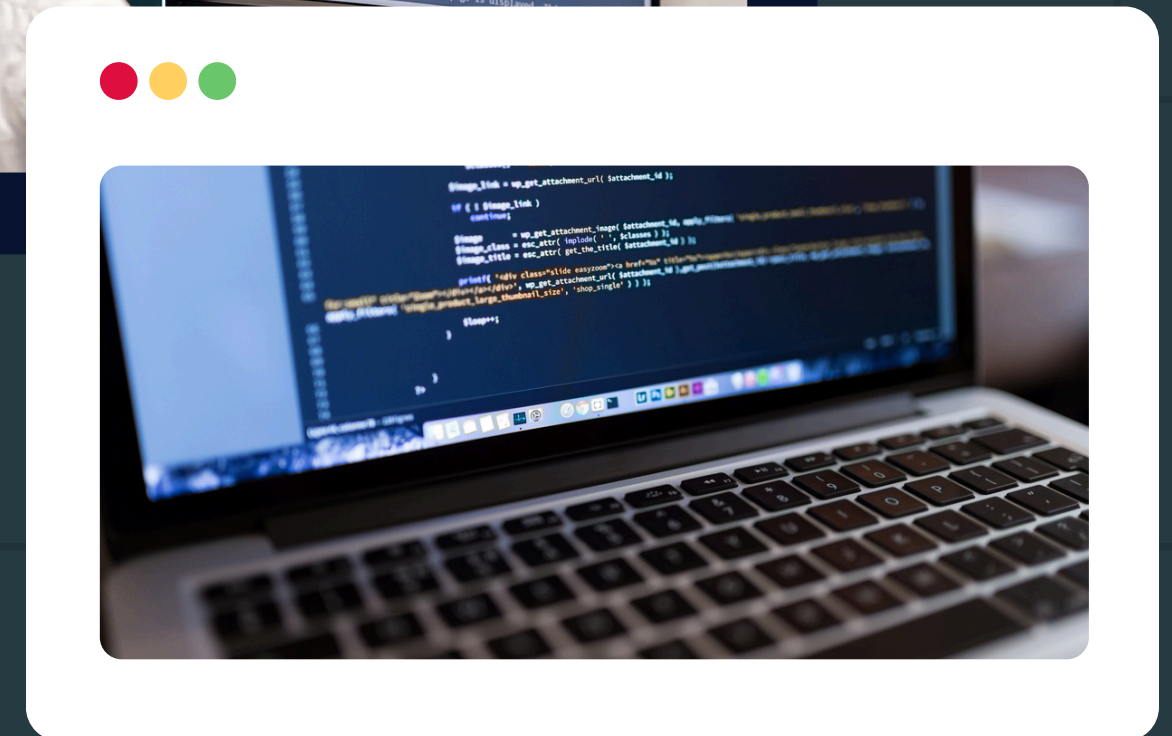
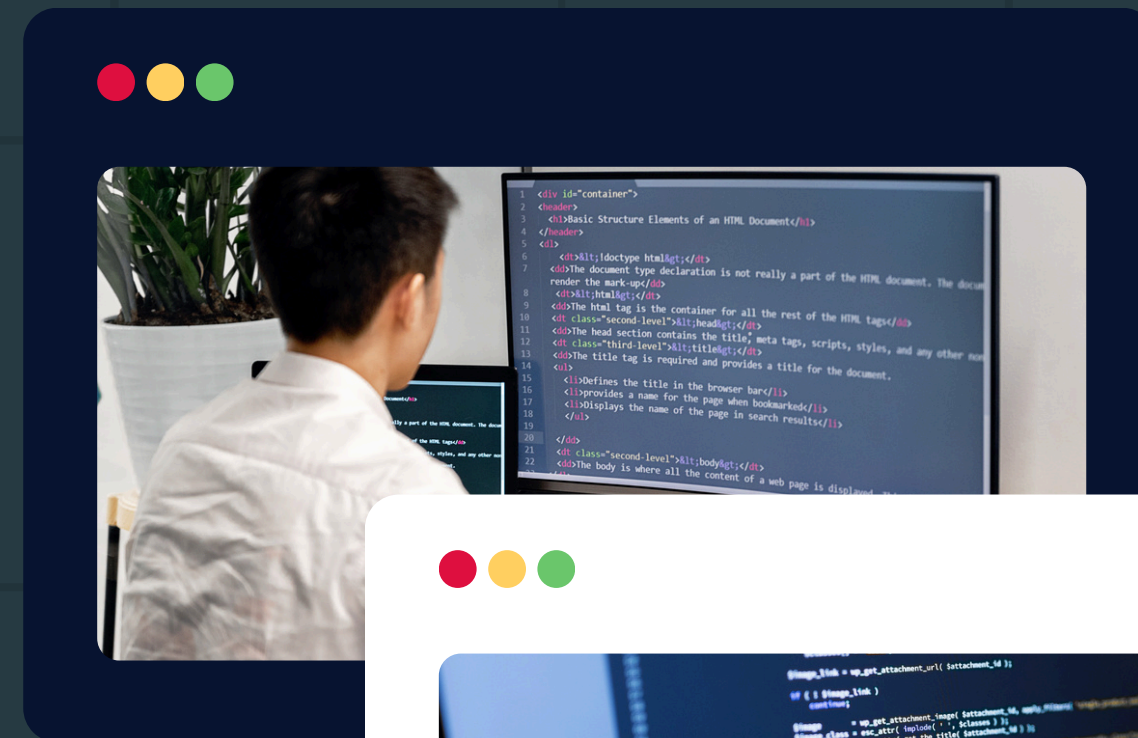


Objetivo : Desenvolver uma aplicação Java capaz de processar um grande volume de dados geográficos e determinar quais são as duas cidades mais distantes dentro de um critério de filtragem.

Manipulação de Dados: O programa deve ler um arquivo CSV (cidades.csv) contendo Continente, País, Cidade, Latitude, Longitude e População.

Matemática: O cálculo da distância deveria considerar a curvatura da Terra, utilizando a Fórmula de Haversine.

Interatividade: O usuário deve ser capaz de filtrar os dados (por Continente, País ou População) via linha de comando.



# Estrutura do Programa



```
import java.util.Scanner;

public class Distancias {
    //variáveis globais

    public static void main (String[] args) {
        ...

        carregarDados();

        ...

        //loop for para comparar as cidades e invocar
        o método calcularDistancia
    }

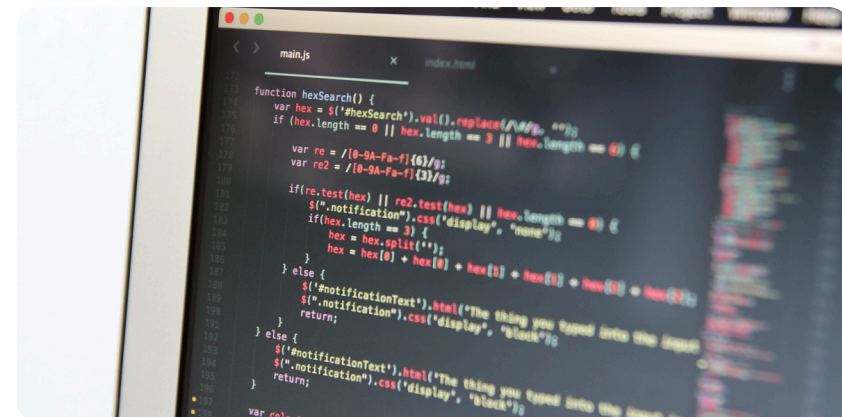
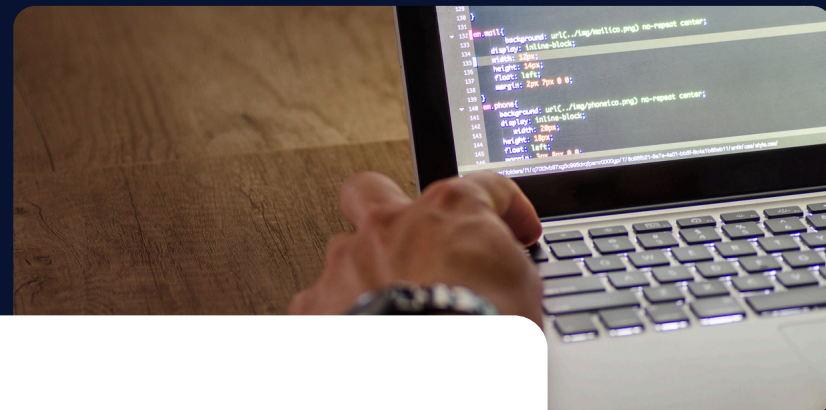
    public static void carregarDados() {
        ...
    }

    public static double calcularDistancia
    (double lat1, double lon1, double lat2,
    double lon2) {
        ...

    }
}
```



# Variáveis globais



```
//tamanho fixo e seguro para os arrays
static final int MAX = 10000;
```

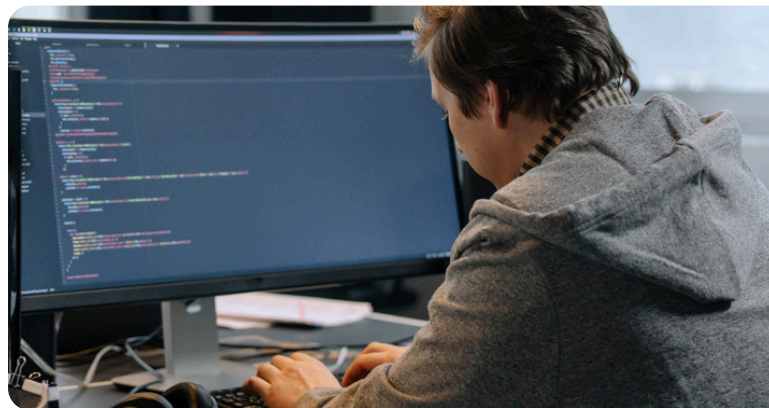
```
//arrays para armazenar cada coluna do arquivo separadamente
static String[] continentes = new String[MAX];
static String[] paises = new String[MAX];
static String[] cidades = new String[MAX];
```

```
static double[] latitudes = new double[MAX];
static double[] longitudes = new double[MAX];
static int[] populacoes = new int[MAX];
```

```
//Variavel para contar quantas cidades realmente guardamos
static int totalCidades = 0;
```

```
//definindo os filtros
static char tipoFiltro = 'N';
static String filtroTexto = "";
static int filtroNumero = 0;
```

# Método Main



```
public static void main (String[] args) {
    if (args.length == 0) {
        tipoFiltro = 'N';
    }
    else if (args.length == 2) {
        tipoFiltro = args[0].charAt(index: 0);

        if (tipoFiltro == '+' || tipoFiltro == '-') {
            filtroNumero = Integer.parseInt(args[1]);
        } else {
            filtroTexto = args[1];
        }
    }

    //metodo para leitura
    carregarDados();

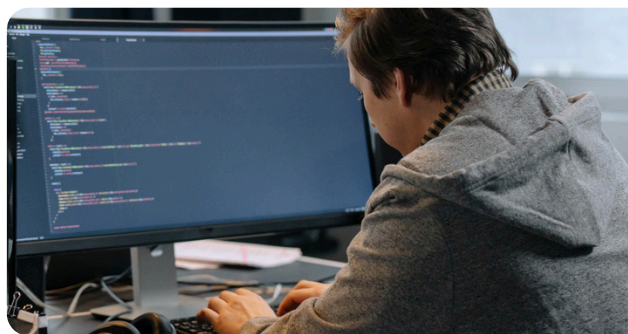
    if (totalCidades < 2) {
        System.out.println(x: "Não existem cidades suficientes.");
    }
    else {
        double maiorDistância = 0.0;
        String cidadeA = "";
        String cidadeB = "";

        //loop duplo, compara todas as cidades contra todas
        for(int i = 0; i < totalCidades; i++) {
            for(int j = i + 1; j < totalCidades; j++) {

                double dist = calcularDistancia(
                    latitudes[i], longitudes[i],
                    latitudes[j], longitudes[j]
                );

                if (dist > maiorDistância) {
                    maiorDistância = dist;
                    cidadeA = cidades[i];
                    cidadeB = cidades[j];
                }
            }
        }
        System.out.printf(format: "As cidades mais distantes são: %s e %s, com distância de %.2f km\n",
                               cidadeA, cidadeB, maiorDistância);
    }
}
```

# Método carregarDados



```
public static void carregarDados() {  
    Scanner sc = new Scanner(System.in);  
    //enquanto houver linhas e não estourar o vetor  
    while (sc.hasNextLine() && totalCidades<MAX) {  
        String linha = sc.nextLine();  
        String[] pedacos = linha.split(regex: ";");  
  
        if (pedacos.length == 6) {  
            String cont = pedacos[0];  
            String pais = pedacos[1];  
            String cid = pedacos[2];  
  
            //conversões do texto para número  
            double lat = Double.parseDouble(pedacos[3]);  
            double lon = Double.parseDouble(pedacos[4]);  
            int pop = Integer.parseInt(pedacos[5]);  
  
            //lógica do filtro  
            boolean deveSalvar = false;  
  
            if (tipoFiltro == 'N') {  
                deveSalvar = true;  
            }  
            else if (tipoFiltro == 'C') {  
                if (cont.equalsIgnoreCase(filtroTexto)) deveSalvar = true;  
            }  
            else if (tipoFiltro=='P') {  
                if(pais.equalsIgnoreCase(filtroTexto)) deveSalvar = true;  
            }  
            else if (tipoFiltro == '+') {  
                if (pop>= filtroNumero) deveSalvar = true;  
            }  
            else if (tipoFiltro == '-') {  
                if (pop <= filtroNumero) deveSalvar = true;  
            }  
  
            // se passou no filtro  
            if (deveSalvar) {  
                continentes[totalCidades] = cont;  
                paises[totalCidades] = pais;  
                cidades[totalCidades] = cid;  
                latitudes[totalCidades] = lat;  
                longitudes[totalCidades] = lon;  
                populacoes[totalCidades] = pop;  
  
                totalCidades++;  
            }  
        }  
    }  
    sc.close();  
}
```

# Método calcularDistancia

```
public static double calcularDistancia (double lat1, double lon1, double lat2, double lon2) {  
  
    //raio da terra  
    final double R = 6378.13;  
  
    //conversões para radianos  
    double lat1Rad = Math.toRadians(lat1);  
    double lat2Rad = Math.toRadians(lat2);  
    double lon1Rad = Math.toRadians(lon1);  
    double lon2Rad = Math.toRadians(lon2);  
  
    double deltaLat = (lat2Rad - lat1Rad);  
    double deltaLon = (lon2Rad - lon1Rad);  
  
    //Fórmula de Haversine  
    double a = Math.pow(Math.sin(deltaLat / 2), 2) +  
                Math.cos(lat1Rad)*Math.cos(lat2Rad) *  
                Math.pow(Math.sin(deltaLon / 2), 2);  
  
    double c = 2 * Math.asin(Math.sqrt(a));  
  
    return (R * c);  
}
```